STUDENT IMPLEMENTATION

Student Record and Academic Management System

Data Organization using Python

Student Details	Value
Student	BANDARU CHANDU
Program	B.Tech AIML
Year	FY AIML - A
Division	B
University	Sanjivani University
PNR	2126UMLM1263

Complete assignment based strictly on the uploaded Student Implementation brief.

1. Objective

The objective of the student project is to apply Python's built-in data structures to develop a Student Record Management System capable of storing, organizing, updating, and retrieving academic information efficiently.

The required workflow is followed in order: problem understanding → algorithm design → flowchart → pseudocode → Python implementation → testing → documentation.

2. Problem Analysis

Requirement	Inputs	Expected Output	Suitable Data Structure
Student information	Roll number, registration number, name, department, subjects, marks, attendance, DOB, contact	Complete student profile	Dictionary + required collections
Add new student	New student details	Added record or duplicate warning	List + Dictionary
Search by roll number	Roll number	Matching record or not found	List + Dictionary
Update information	Roll number, field, new value	Updated record	Dictionary
Delete record	Roll number	Record removed	List
Calculate average marks	Marks	Average	List
Find highest scorer	All student records	Highest average scorer	List + Dictionary
List by department	Department	Matching records	List + Dictionary
Count students	Student collection	Total number	List
Generate reports	Student collection	Academic summary	Sets + List + Dictionary

3. Data Structure Selection Report

Data Structure	Required Use	Reason
Strings	Student names, departments, addresses, email IDs	Text data is represented as strings.
Lists	Student records, subject lists, marks, attendance	Ordered, changeable collections can grow or shrink.
Tuples	Roll number, registration number, date of birth	Fixed values are stored as immutable tuples.
Sets	Departments, subjects, student clubs	Unique values are needed; duplicates are removed automatically.
Dictionaries	Complete student profile	Named key-value fields make records easy to access and update.

4. Algorithm Design

4.1 Add New Student

1. Accept student information.

2. Check whether the roll number already exists.

3. If it exists, reject the duplicate record.

4. Otherwise create the complete student dictionary using the required collections.

5. Append the record to the student list.

6. Display confirmation.

4.2 Search Student by Roll Number

1. Accept the roll number.

2. Compare it with each stored record.

3. If a match is found, display/return the record.

4. If no match is found, display 'Student not found'.

4.3 Update Student Information

1. Accept the roll number.

2. Search for the record.

3. If not found, display 'Student not found'.

4. Read the field and new value.

5. Update the selected field.

6. Display confirmation.

4.4 Delete Student Record

1. Accept the roll number.

2. Search for the record.

3. If not found, display 'Student not found'.

4. Remove the record from the list.

5. Display confirmation.

4.5 Calculate Average Marks

1. Read the marks list.

2. If empty, return 0.

3. Add the marks and divide by the number of marks.

4. Return the average.

4.6 Find Highest Scorer

1. Check whether records exist.

2. Calculate the average marks of each student.

3. Select the student with the highest average.

4. Display the highest scorer.

4.7 List Students by Department

1. Accept the department.
2. Convert it to a consistent form.
3. Check every student.
4. Collect matching records.
5. Display the result.

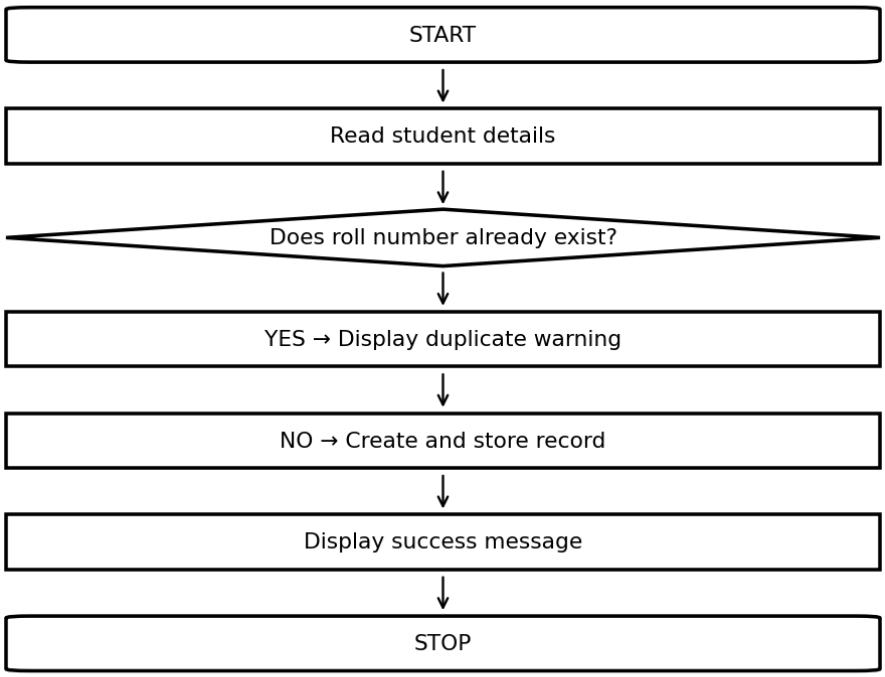
4.8 Count Students

1. Read the student collection.
2. Count its records.
3. Display the total.

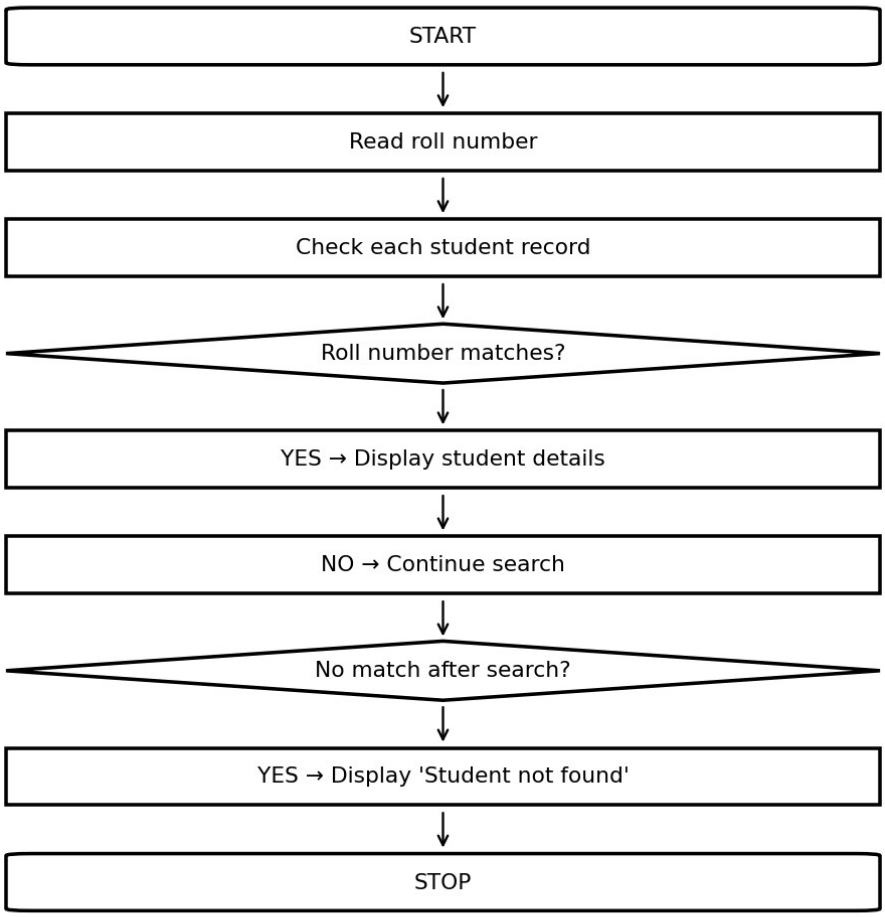
5. Flowchart Collection

The following diagrams cover exactly the seven flowchart operations requested in the assignment.

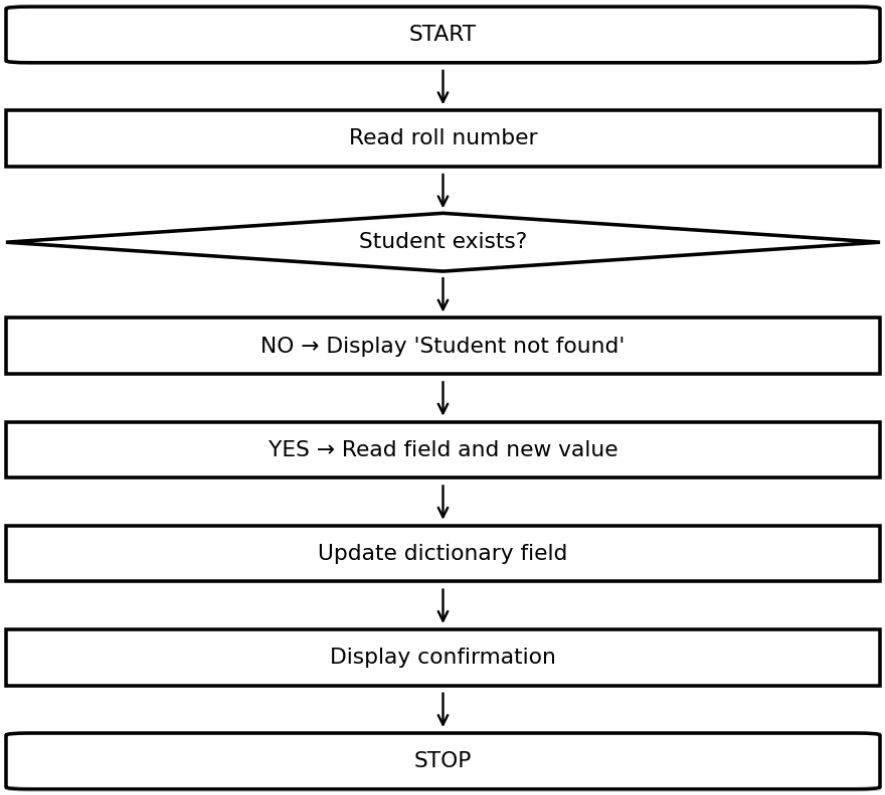
5.1 Add Student



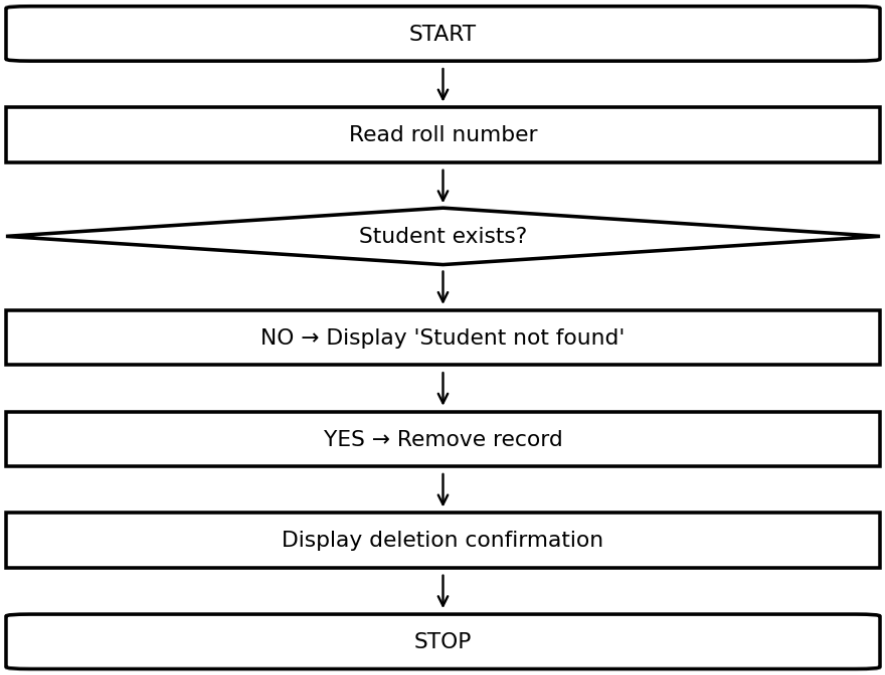
5.2 Search Student



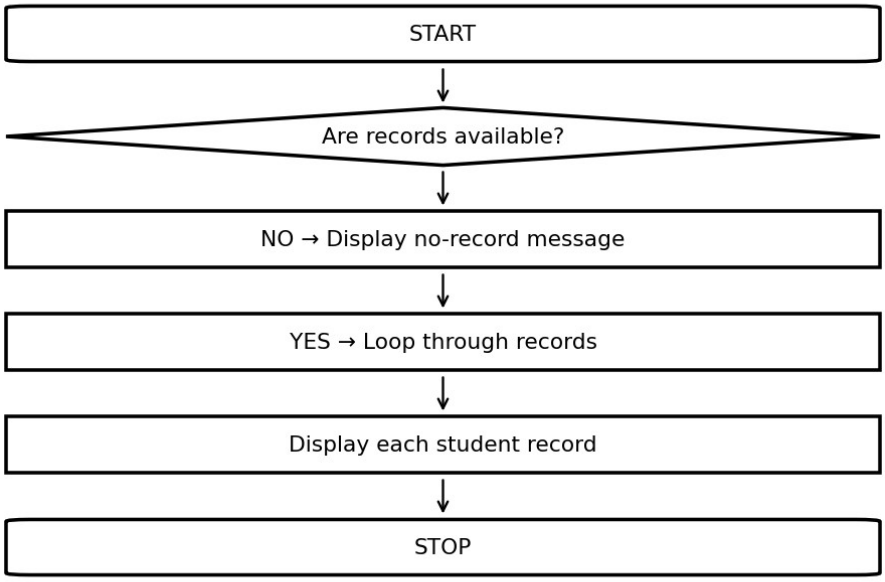
5.3 Update Record



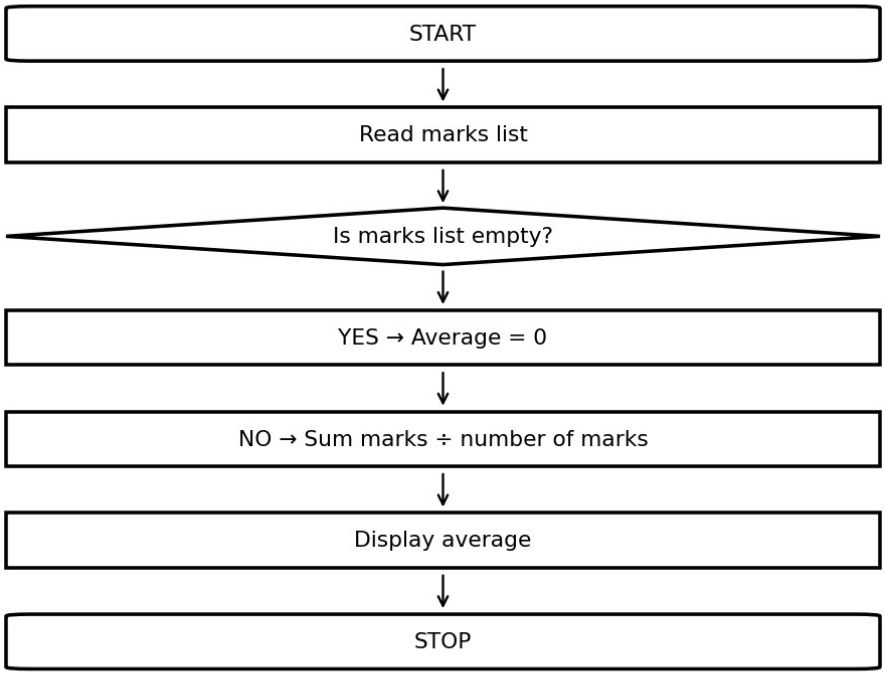
5.4 Delete Record



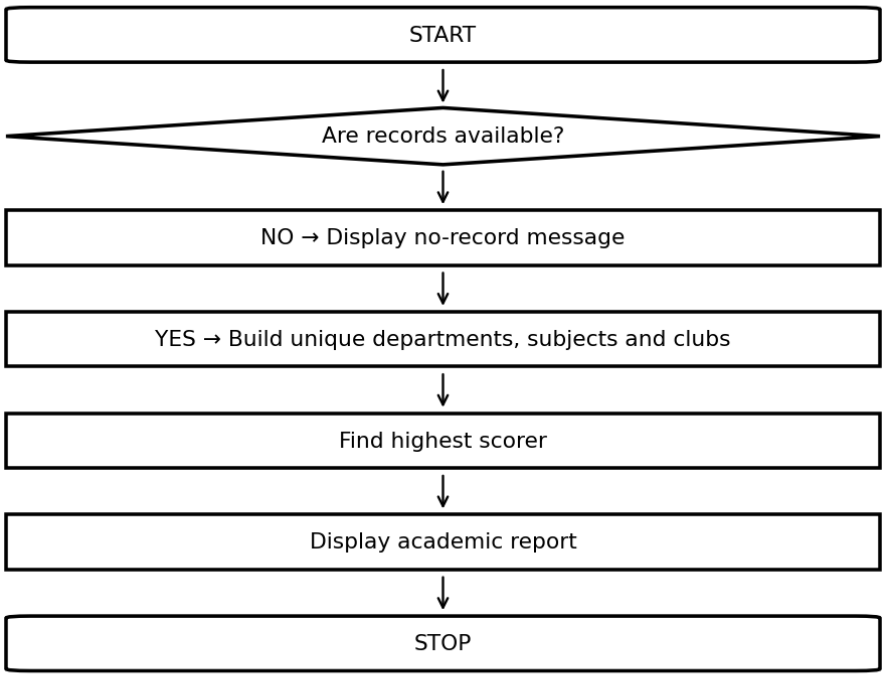
5.5 Display Records



5.6 Calculate Average



5.7 Generate Reports



6. Pseudocode Document

Add Student

```
START
READ student details
IF roll number already exists THEN
   DISPLAY "Student already exists"
ELSE
   CREATE student dictionary
   STORE roll number, registration number and DOB as tuples
   STORE subjects, marks and attendance as lists
   STORE clubs as a set
   APPEND dictionary to student collection
   DISPLAY "Student added successfully"
END IF
STOP
```

Search Student

```
START
READ roll number
SET found = False
FOR each student in collection
   IF roll number matches THEN
      DISPLAY student details
      SET found = True
      EXIT LOOP
   END IF
END FOR
IF found == False THEN
   DISPLAY "Student not found"
END IF
STOP
```

Update Record

```
START
READ roll number
SEARCH for student
IF student does not exist THEN
   DISPLAY "Student not found"
ELSE
   READ field and new value
   UPDATE selected field
   DISPLAY confirmation
END IF
STOP
```

Delete Record

```
START
READ roll number
SEARCH for student
IF student does not exist THEN
   DISPLAY "Student not found"
ELSE
   REMOVE student from collection
   DISPLAY confirmation
```

```
END IF
STOP
```

Display Records

```
START
IF collection is empty THEN
   DISPLAY "No student records available"
ELSE
   FOR each student
      DISPLAY all fields
      DISPLAY average marks
   END FOR
END IF
STOP
```

Calculate Average

```
START
READ marks
IF marks list is empty THEN
   SET average = 0
ELSE
   SET average = SUM(marks) / NUMBER OF marks
END IF
DISPLAY average
STOP
```

Generate Reports

```
START
IF collection is empty THEN
   DISPLAY "No records available for report"
ELSE
   CREATE unique department, subject and club sets
   FIND highest scorer
   COUNT students
   DISPLAY report
END IF
STOP
```

Find Highest Scorer

```
START
IF collection is empty THEN
   RETURN no result
ELSE
   CALCULATE average for every student
   SELECT maximum average
   DISPLAY highest scorer
END IF
STOP
```

List by Department

```
START
READ department
CREATE empty result list
FOR each student
   IF department matches THEN
      APPEND student to result list
   END IF
```

```
END FOR
DISPLAY result list
STOP
```

Count Students

```
START
SET count = number of records
DISPLAY count
STOP
```

7. String Programming Exercises

```
title = "  python programming  "
print(title.upper())
print(title.lower())
print(title.title())
print(title.capitalize())
print(title.strip())
print("python,java,c".split(","))
print(", ".join(["Python", "Java", "C"]))
print("Dune".replace("u", "o"))
print("library".find("brar"))
print("banana".count("a"))
print("1984".startswith("19"))
print("report.pdf".endswith(".pdf"))

name = "  aarav kumar  "
print(name.strip().title())
```

8. List Management Programs

```
books = ["1984", "Dune", "Emma"]
books.append("Atomic Habits")
books.insert(0, "Python Basics")
books.extend(["Clean Code", "The Alchemist"])
books.remove("Dune")
removed = books.pop(0)
books.sort()
print(books.index("Emma"))
print(books.count("Emma"))
books.reverse()
temporary = ["A", "B"]
temporary.clear()
print(temporary)
```

9. Tuple-Based Programs

```
roll_number = ("S001",)
registration_number = ("REG001",)
dob = ("2006", "04", "19")

roll, = roll_number
reg, = registration_number
year, month, day = dob
```

```
print(roll, reg, year, month, day)
# roll_number[0] = "S002"  # TypeError: tuple is immutable
```

10. Set Operations Programs

```
a = {"CSE", "AIML", "ECE"}
b = {"AIML", "EEE", "ECE"}

print("Union:", a | b)
print("Intersection:", a & b)
print("Difference:", a - b)

values = ["CSE", "AIML", "CSE", "ECE"]
unique_values = set(values)
print("Unique:", unique_values)

unique_values.add("EEE")
unique_values.discard("ECE")
```

11. Dictionary-Based Student Record Management System

```
student = {
   "roll_number": ("S001",),
   "registration_number": ("REG001",),
   "name": "Aarav Kumar",
   "department": "CSE",
   "subjects": ["Python", "Mathematics"],
   "marks": [86, 94],
   "attendance": [92, 95],
   "dob": ("2006", "04", "19"),
   "contact": {
      "email": "aarav@example.com",
      "address": "Guntur"
   },
   "clubs": {"AI Club", "Coding Club"}
}

print(student.keys())
print(student.get("name"))
student.update({"department": "AIML"})
print(student)
```

12. Data Structure Integration

The complete application integrates all five required collections. A list stores multiple student dictionaries. Each dictionary contains strings, lists, tuples, sets and contact information. Reports use sets to obtain unique departments, subjects and clubs.

Structure	Integrated Use
String	Name, department, address, email, messages
List	Students, subjects, marks, attendance
Tuple	Roll number, registration number, date of birth
Set	Departments/subjects/clubs in reports; student clubs in records
Dictionary	Complete student profile and contact information

12.1 Complete Python Implementation

```
"""

Student Record and Academic Management System

Data Organization using Python



B.Tech AIML | FY AIML - A | Division A

Sanjivani University

"""



students = []




def clean_text(value):

    if isinstance(value, tuple):

        return value[0].strip() if value else ""

    return str(value).strip() if value is not None else ""




def add_student(roll_number, registration_number, name, department,

                subjects, marks, attendance, dob, email, address, clubs):

    """Add a new student record after checking for duplicate roll numbers."""
```

```
roll_number = (clean_text(roll_number).upper(),)        # Tuple
registration_number = (clean_text(registration_number).upper(),) # Tuple
name = clean_text(name).title()                         # String
department = clean_text(department).upper()             # String
email = clean_text(email).lower()                       # String
address = clean_text(address)                           # String
dob = tuple(clean_text(dob).split("-"))                 # Tuple


for student in students:
    if student["roll_number"] == roll_number:
        print("Student already exists.")
        return False


student = {
    "roll_number": roll_number,
    "registration_number": registration_number,
    "name": name,
    "department": department,
    "subjects": list(subjects),              # List
    "marks": list(marks),                    # List
    "attendance": list(attendance),          # List
    "dob": dob,
```

```
        "contact": {"email": email, "address": address},
        "clubs": set(clubs)                    # Set
    }

    students.append(student)                   # List of dictionaries
    print("Student added successfully.")
    return True


def search_student(roll_number):
    """Search for a student by roll number."""
    roll_number = (clean_text(roll_number).upper(),)
    for student in students:
        if student["roll_number"] == roll_number:
            return student
    return None


def update_student(roll_number, field, new_value):
    """Update an allowed student field."""
    student = search_student(roll_number)
    if student is None:
```

```
    print("Student not found.")
    return False

field = clean_text(field).lower()

if field == "roll_number":
    new_roll = (clean_text(new_value).upper(),)
    for s in students:
        if s is not student and s["roll_number"] == new_roll:
            print("Error: Roll number already exists for another student.")
            return False
    student["roll_number"] = new_roll
elif field == "registration_number":
    student["registration_number"] = (clean_text(new_value).upper(),)
elif field == "name":
    student["name"] = clean_text(new_value).title()
elif field == "department":
    student["department"] = clean_text(new_value).upper()
elif field == "email":
    student["contact"]["email"] = clean_text(new_value).lower()
elif field == "address":
    student["contact"]["address"] = clean_text(new_value)
```

```
    elif field == "dob":
        student["dob"] = tuple(clean_text(new_value).split("-"))
    elif field == "subjects":
        student["subjects"] = list(new_value)
    elif field == "attendance":
        student["attendance"] = list(new_value)
    elif field == "marks":
        student["marks"] = list(new_value)
    elif field == "clubs":
        student["clubs"] = set(new_value)
    else:
        print("Invalid field.")
        return False

    print("Student record updated successfully.")
    return True



def delete_student(roll_number):
    """Delete a student record by roll number."""
    student = search_student(roll_number)
    if student is None:
```

```
        print("Student not found.")
        return False
    students.remove(student)
    print("Student record deleted successfully.")
    return True



def calculate_average(student):
    """Calculate the average of a student's marks."""
    if not student:
        return 0.0
    marks = student.get("marks", [])
    if not marks:
        return 0.0
    return sum(marks) / len(marks)



def find_highest_scorer():
    """Return the student with the highest average marks."""
    if not students:
        return None
    return max(students, key=calculate_average)
```

```
def list_by_department(department):
    """Return all students belonging to a department."""
    department = clean_text(department).upper()
    return [student for student in students
            if student["department"] == department]



def count_students():
    return len(students)



def display_records(records=None):
    """Display all records or a supplied collection."""
    records = students if records is None else records
    if not records:
        print("No student records available.")
        return

    print("\n" + "=" * 72)
    print("STUDENT RECORDS")
```

```
    print("=" * 72)



    for student in records:

        print(f"Roll Number : {student['roll_number'][0]}")

        print(f"Registration: {student['registration_number'][0]}")

        print(f"Name        : {student['name']}")

        print(f"Department  : {student['department']}")

        print(f"Subjects    : {', '.join(student['subjects'])}")

        print(f"Marks       : {student['marks']}")

        print(f"Average     : {calculate_average(student):.2f}")

        print(f"Attendance  : {student['attendance']}")

        print(f"DOB         : {'-'.join(student['dob'])}")

        print(f"Email       : {student['contact']['email']}")

        print(f"Address     : {student['contact']['address']}")

        print(f"Clubs       : {', '.join(sorted(student['clubs']))}")

        print("-" * 72)



def generate_report():

    """Generate a summary report using sets."""

    if not students:

        print("No records available for report.")
```

```
    return


departments = {student["department"] for student in students}

subjects = {subject for student in students for subject in student["subjects"]}

clubs = {club for student in students for club in student["clubs"]}


print("\n" + "=" * 72)

print("ACADEMIC REPORT")

print("=" * 72)

print(f"Total students      : {count_students()}")

print(f"Departments         : {', '.join(sorted(departments))}")

print(f"Unique subjects     : {', '.join(sorted(subjects))}")

print(f"Student clubs       : {', '.join(sorted(clubs))}")


top = find_highest_scorer()

if top:

    print(f"Highest scorer      : {top['name']} ({calculate_average(top):.2f})")


print("\nStudents by department:")

for department in sorted(departments):

    print(f"  {department}: {len(list_by_department(department))}")
```

```
def seed_demo_data():
    add_student("S001", "REG001", "Aarav Kumar", "CSE",
                ["Python", "Mathematics", "Physics"], [86, 91, 84], [92, 95, 90],
                "2006-04-19", "aarav@example.com", "Guntur, Andhra Pradesh",
                ["Coding Club", "AI Club"])
    add_student("S002", "REG002", "Meera Sharma", "AIML",
                ["Python", "Mathematics", "AI"], [94, 89, 96], [97, 91, 95],
                "2006-08-12", "meera@example.com", "Vijayawada, Andhra Pradesh",
                ["AI Club"])



def main():
    seed_demo_data()


    while True:
        print("\n" + "=" * 72)
        print("STUDENT RECORD AND ACADEMIC MANAGEMENT SYSTEM")
        print("=" * 72)
        print("1. Add Student")
        print("2. Search Student")
        print("3. Update Student")
```

```
print("4. Delete Student")

print("5. Display Records")

print("6. Calculate Average")

print("7. Generate Reports")

print("8. Find Highest Scorer")

print("9. List Students by Department")

print("10. Count Students")

print("0. Exit")



choice = input("Enter your choice: ").strip()



if choice == "1":

    roll = input("Roll number: ")

    reg = input("Registration number: ")

    name = input("Name: ")

    dept = input("Department: ")

    subjects = [s.strip() for s in input("Subjects (comma-separated): ").split(",") if s.strip()]

    marks_input = input("Marks (comma-separated): ")

    attendance_input = input("Attendance (comma-separated): ")

    try:

        marks = [float(x.strip()) for x in marks_input.split(",") if x.strip()]

        attendance = [float(x.strip()) for x in attendance_input.split(",") if x.strip()]
```

```
        except ValueError:
            print("Error: Marks and attendance must be numbers.")
            continue
        dob = input("Date of birth (YYYY-MM-DD): ")
        email = input("Email: ")
        address = input("Address: ")
        clubs = [c.strip() for c in input("Clubs (comma-separated): ").split(",") if c.strip()]
        add_student(roll, reg, name, dept, subjects, marks, attendance, dob, email, address, clubs)

    elif choice == "2":
        result = search_student(input("Enter roll number: "))
        if result:
            display_records([result])
        else:
            print("Student not found.")

    elif choice == "3":
        roll = input("Enter roll number: ")
        field = input("Field (roll_number/registration_number/name/department/email/address/dob/
subjects/marks/attendance/clubs): ").strip().lower()
        if field in {"marks", "attendance"}:
            try:
                values = [float(v.strip()) for v in input("Enter comma-separated values: ").split(",") if v.strip()]
```

```
            update_student(roll, field, values)
        except ValueError:
            print("Error: Marks and attendance must be numbers.")
    elif field in {"subjects", "clubs"}:
        values = [v.strip() for v in input("Enter comma-separated values: ").split(",") if v.strip()]
        update_student(roll, field, values)
    else:
        update_student(roll, field, input("New value: "))

elif choice == "4":
    delete_student(input("Enter roll number: "))

elif choice == "5":
    display_records()

elif choice == "6":
    result = search_student(input("Enter roll number: "))
    if result:
        print(f"Average marks: {calculate_average(result):.2f}")
    else:
        print("Student not found.")
```

```
elif choice == "7":
    generate_report()

elif choice == "8":
    top = find_highest_scorer()
    if top:
        print(f"Highest scorer: {top['name']} - {calculate_average(top):.2f}")
    else:
        print("No student records available.")

elif choice == "9":
    dept = input("Enter department: ")
    records = list_by_department(dept)
    if records:
        display_records(records)
    else:
        print(f"No students found in department '{dept.strip().upper()}'.")

elif choice == "10":
    print(f"Total students: {count_students()}")

elif choice == "0":
```

```
            print("Thank you. Program ended.")
            break

        else:
            print("Invalid choice. Please try again.")


if __name__ == "__main__":
    main()
```

13. Testing Report

ID	Test Type	Action	Expected Result	Status
T01	Manual/Normal	Add valid student	Record added	PASS
T02	Search	Search existing S001	Correct record displayed	PASS
T03	Invalid	Search S999	Student not found; no crash	PASS
T04	Duplicate	Add S001 again	Duplicate rejected	PASS
T05	Update	Change S001 email	Email updated	PASS
T06	Delete	Delete S002	Record removed	PASS
T07	Empty	Highest scorer with empty list	No result; no crash	PASS
T08	Boundary	Average of empty marks	0.0 returned	PASS
T09	Department	List CSE students	Only matching records	PASS
T10	Count	Count after deletion	Correct total	PASS
T11	Report	Generate report	Unique sets + scorer displayed	PASS

13.1 Dry-Run / Trace Table

Step	Action	Result
1	Start with empty student list	[]
2	Add S001	No duplicate
3	Store record	One dictionary appended
4	Calculate average	(86+94)/2 = 90.00

5	Search S001	Aarav Kumar found
6	Try duplicate S001	Rejected

13.2 Debugging / Verification

The supplied Python implementation was executed with automated assertions covering adding, searching, updating, deleting, average calculation, highest scorer, department filtering, counting, duplicate rejection and empty-data behavior.

Automated verification: ALL TESTS PASSED

14. Observations

- The list provides a changeable collection of student records.
- Each dictionary gives the student profile named fields instead of position-based values.
- Lists are appropriate for subjects, marks and attendance.
- Tuples keep roll number, registration number and date of birth fixed after creation.
- Sets provide unique department, subject and club values for reporting.
- Strings handle names, departments, addresses, emails and messages.
- The integrated design is easier to update and test than separate variables for every student.

15. Screenshots

The assignment explicitly asks for screenshots. Two representative program-output screenshots are included below. They are generated from the same program outputs shown in the implementation.

Sample VS Code / Terminal Output — Calculate Average

```
STUDENT RECORD AND ACADEMIC MANAGEMENT SYSTEM
1. Add Student   2. Search Student   3. Update Student
4. Delete Student   5. Display Records   6. Calculate Average
7. Generate Reports   8. Find Highest Scorer   9. Department
10. Count Students   0. Exit

Enter your choice: 6
Enter roll number: S001
Average marks: 90.00
```

Sample VS Code / Terminal Output — Academic Report

```
ACADEMIC REPORT
Total students      : 2
Departments         : AIML, CSE
Unique subjects     : AI, Mathematics, Physics, Python
Student clubs       : AI Club, Coding Club
Highest scorer      : Meera Sharma (93.00)

Students by department:
  AIML: 1
  CSE : 1
```

16. README

Project: Student Record and Academic Management System – Data Organization using Python

Purpose: Store, organize, update and retrieve student academic information using Python's built-in data structures.

HOW TO RUN
1. Install Python 3.
2. Open the project folder in VS Code.
3. Open Terminal.
4. Run:
   python student_record_management_system.py
5. Use the numbered menu to test each operation.

The program starts with two demonstration records so that the main operations
can be tested immediately.

17. Student Deliverables Checklist

☐ Problem Analysis Report

☐ Data Structure Selection Report

☐ Algorithm Design Document

☐ Flowchart Collection

☐ Pseudocode Document

☐ String Programming Exercises

☐ List Management Programs

☐ Tuple-Based Programs

☐ Set Operations Programs

□ Dictionary-Based Student Record Management System

□ Integrated Student Record Management Application

□ Testing Report

□ Project Documentation

□ Updated GitHub Repository